make that AppImage an executable one:

```
$cd Test_Table
$chmod a+x textosaurus-0.9.7-f0908a3-
linux64.AppImage
```

The next step is to create a directory to save the app configuration files:

```
mkdir textosaurus-0.9.7-f0908a3-linux64.
AppImage.home
```

After that, execute the AppImage and make some changes in the defaults (for example, changing the font and its size). Now, close the app.

To ensure that the configuration files are created properly, enter the following command in the console:

```
find textosaurus-0.9.7-f0908a3-linux64.
AppImage.home
```

The above command will display the location of the configuration files. For our example, the output will be like this:

```
textosaurus-0.9.7-f0908a3-linux64.
AppImage.home
textosaurus-0.9.7-f0908a3-linux64.
AppImage.home/.config
textosaurus-0.9.7-f0908a3-linux64.
AppImage.home/.config/Textosaurus
textosaurus-0.9.7-f0908a3-linux64.
AppImage.home/.config/Textosaurus/config.ini
```

Updating AppImages

AppImages use the Zsync2 algorithm for updation. This algorithm downloads the pieces that are changed between your local AppImage and the new AppImage, and could be very useful for the large size AppImages.

To update an AppImage you can either use *AppImageUpdate* or *appimageupdatetoo*l. The difference between the two is that the former is the GUI version and the latter is the console version. Download any one of these according to your needs, make them executable, and run them with the complete path of your AppImage that needs to be updated as an argument.

As an example, if you want to check whether a new version of *AppImageUpdate* is available, you can issue the following command in the console:

```
./AppImageUpdate-x86_64.AppImage /
home/tux_machine/Downloads/
AppImageUpdate-x86_64.AppImage
```

It will open up a progress bar to let you know the overall updation process.

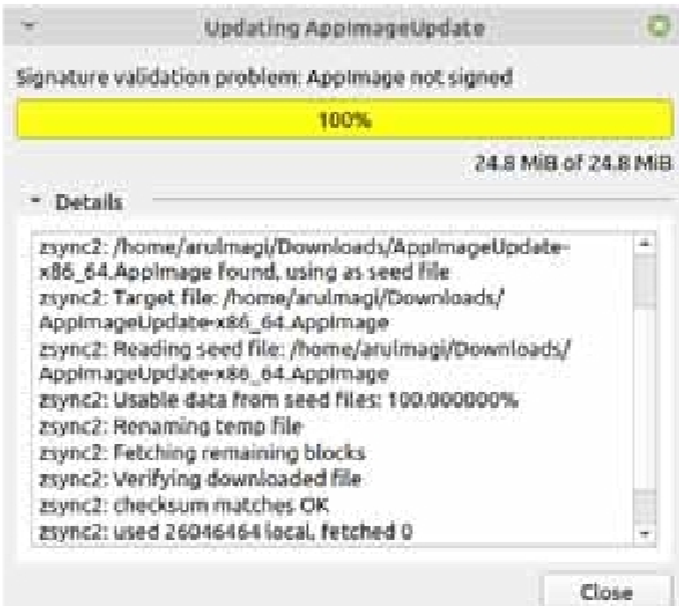

Figure 2: AppImage progress bar

AppImage is fairly young and is evolving, so this article would be incomplete if its pros and cons aren't listed here.

Pros of AppImage

1. Even though an AppImage contains the app along with its supporting files and libraries in a single package, its overall size is kept at a reasonable level by employing *SquashFS*, which is used to compress files, inodes and directories.
2. AppImages is a decentralised system, so it is not controlled by a single entity. Anyone can craft an AppImage for his or her app, and host it in their site or list it in the crowdsourced directory like *AppImageHub*. Currently, it consists of a total of 689 apps.
3. AppImages are capable of running in an isolated environment; so you can run multiple versions of the same app in a system. This is extremely useful when testing multiple versions of the same app.
4. Since installation is not a requirement for AppImages, they can run in Live OS. This comes in handy when using Linux as a Live OS in a Windows installed system. You can use any AppImage in that environment.
5. AppImages are built on the principle of One App = One File. So maintaining the AppImage is fairly easy, especially for the non-tech users.
6. AppImages are entirely open source; so naturally, no proprietary apps are developed in the AppImage format.

Cons of AppImage

1. AppImages remove maintainers from the equation. Hence, it is up to the developers to ensure that the ingredients of the AppImage should not be outdated and be properly vetted. This is additional baggage for developers.
2. The size of an AppImage is higher than what the traditional *.deb* package may offer. For example, the installation of the Flameshot app via the APT method will require packages that are 217KB in size to be downloaded, but the size of the same app in AppImage will be 19.67MB.
3. Apps in the AppImage format are fairly fewer compared to the traditional package managers; however, the number of apps in the AppImage format is gradually increasing.

AppImage is a good replacement for traditional package managers, but it is still a work in progress. Due to its many advantages, it has a lot of potential in the future for developers as well as the Linux community. END

By: K. Magimai Prakash

The author is a freelance writer, FOSS enthusiast, and a big supporter of decentralisation and privacy, among other things.